

DSA Patterns – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Dec 2026 – 20 Dec 2026 (Days 113-126)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Pattern-first preparation with templates, problems, complexity analysis, and interview scripts.

PREREQUISITES – Complete FIRST

PDF	Sections needed
<code>08b-dsa-datastructures-fundamentals-complete.pdf</code>	All §1-10 (stack, queue, LL, tree, heap, graph)
100+ LeetCode easy/medium	Pattern recognition at speed

When: Month 5, Days 113+ (after Month 1–4 LC foundation).

Next after this: `14-advanced-dsa-hard-complete.pdf` (Days 127+).

Table of Contents

1. How to Practice
2. Complexity Guide
3. Pattern 1: Two Pointers
4. Pattern 2: Sliding Window
5. Pattern 3: Hash Map / Set
6. Pattern 4: Binary Search

7. [Pattern 5: BFS & DFS](#)
8. [Pattern 6: Trees](#)
9. [Pattern 7: Graphs](#)
10. [Pattern 8: Dynamic Programming](#)
11. [Pattern 9: Heap / Priority Queue](#)
12. [Pattern 10: Stack / Monotonic Stack](#)
13. [NeetCode 150 Mapping](#)
14. [LeetCode Practice Plan](#)
15. [Interview Script](#)
16. [Product-company DSA \(1 CR\)](#)
17. [Pattern 11: Intervals](#)
18. [Pattern 12: Backtracking](#)

1. How to Practice

Rules

1. **Pattern-first** — master template, then 3–5 problems per pattern
2. **Timed practice** — Easy: 25 min | Medium: 35 min | Hard: 45 min
3. **Explain aloud** — approach, complexity, edge cases, trade-offs
4. **No hints first 15 min** — struggle builds pattern recognition
5. **Track progress** — goal: 150–200 problems by interview

Platforms

Platform	URL	Best For
LeetCode	leetcode.com	Primary practice
NeetCode	neetcode.io	Curated roadmap
AlgoExpert	algoexpert.io	Video explanations

2. Complexity Guide

Big-O Cheat Sheet

n	$O(n)$	$O(n \log n)$	$O(n^2)$
100	instant	instant	instant
1,000	instant	instant	~1ms
10,000	instant	instant	~100ms
100,000	~1ms	~2ms	TLE
1,000,000	~10ms	~20ms	TLE

Common Complexities by Pattern

Pattern	Time	Space
Two Pointers	$O(n)$	$O(1)$
Sliding Window	$O(n)$	$O(k)$
Hash Map	$O(n)$	$O(n)$
Binary Search	$O(\log n)$	$O(1)$
BFS/DFS	$O(V+E)$	$O(V)$
DP	$O(n^2)$ or $O(n)$	$O(n)$ or $O(n^2)$
Heap (k elements)	$O(n \log k)$	$O(k)$

Space-Time Tradeoffs

- Hash map: $O(n)$ space for $O(1)$ lookup
- Sorting first: $O(n \log n)$ enables two pointers
- Memoization: $O(n)$ space avoids exponential time

3. Pattern 1: Two Pointers

When to use: Sorted arrays, palindromes, pairs, in-place removal, merging sorted arrays.

Template

```
function twoPointers(nums: number[]): number {
  let left = 0;
  let right = nums.length - 1;

  while (left < right) {
    const sum = nums[left] + nums[right];
    if (sum === target) return /* found */;
    if (sum < target) left++;
    else right--;
  }
  return -1;
}
```

Fast & Slow (Linked List)

```
function hasCycle(head: ListNode | null): boolean {
  let slow = head, fast = head;
  while (fast?.next) {
    slow = slow!.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	<u>Two Sum II</u>	Easy	Sorted → two ends
2	<u>3Sum</u>	Medium	Fix one, two-pointer rest
3	<u>Container With Most Water</u>	Medium	Move shorter side
4	<u>Valid Palindrome</u>	Easy	Skip non-alnum
5	<u>Remove Duplicates from Sorted Array</u>	Easy	Write pointer

4. Pattern 2: Sliding Window

When to use: Contiguous subarray/substring, fixed or variable window size, frequency constraints.

Fixed Window Template

```
function fixedWindow(arr: number[], k: number): number {
  let windowSum = 0, maxSum = 0;
  for (let i = 0; i < arr.length; i++) {
    windowSum += arr[i];
    if (i >= k - 1) {
      maxSum = Math.max(maxSum, windowSum);
      windowSum -= arr[i - k + 1];
    }
  }
  return maxSum;
}
```

Variable Window Template

```
function variableWindow(s: string): number {
  const freq = new Map<string, number>();
  let left = 0, maxLen = 0;

  for (let right = 0; right < s.length; right++) {
    freq.set(s[right], (freq.get(s[right]) ?? 0) + 1);

    while (/* window invalid */) {
      freq.set(s[left], freq.get(s[left])! - 1);
      left++;
    }
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	Maximum Average Subarray I	Easy	Fixed k window
2	Longest Substring Without Repeating Characters	Medium	Shrink on duplicate
3	Minimum Window Substring	Hard	Track required chars
4	Permutation in String	Medium	Fixed window + freq match
5	Max Consecutive Ones III	Medium	At most k zeros

5. Pattern 3: Hash Map / Set

When to use: O(1) lookup, frequency counting, grouping, duplicate detection.

Template

```
function hashMapPattern(nums: number[], target: number): number[] {
  const map = new Map<number, number>();
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (map.has(complement)) return [map.get(complement)!, i];
    map.set(nums[i], i);
  }
  return [];
}
```

Frequency Counter

```
function topKFrequent(nums: number[], k: number): number[] {
  const freq = new Map<number, number>();
  for (const n of nums) freq.set(n, (freq.get(n) ?? 0) + 1);
  return [...freq.entries()]
    .sort((a, b) => b[1] - a[1])
    .slice(0, k)
    .map(([n]) => n);
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	Two Sum	Easy	Complement lookup
2	Group Anagrams	Medium	Sort string as key
3	Top K Frequent Elements	Medium	Map + heap/bucket
4	Longest Consecutive Sequence	Medium	Set, check sequence start
5	Valid Anagram	Easy	Char frequency

6. Pattern 4: Binary Search

When to use: Sorted data, find boundary, minimize/maximize with monotonic condition, search space reduction.

Template

```
function binarySearch(nums: number[], target: number): number {
  let lo = 0, hi = nums.length - 1;
  while (lo <= hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] === target) return mid;
    if (nums[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}
```

Binary Search on Answer

```
function minCapacity(weights: number[], days: number): number {
  let lo = Math.max(...weights);
  let hi = weights.reduce((a, b) => a + b, 0);

  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (canShip(weights, days, mid)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	<u>Binary Search</u>	Easy	Classic template
2	<u>Search Insert Position</u>	Easy	Return lo
3	<u>Find Minimum in Rotated Sorted Array</u>	Medium	Compare mid vs right
4	<u>Koko Eating Bananas</u>	Medium	BS on answer
5	<u>Median of Two Sorted Arrays</u>	Hard	Partition both arrays

7. Pattern 5: BFS & DFS

When to use: Graphs, trees, grids, level-order, connected components, shortest path (unweighted).

BFS Template

```
function bfs(grid: string[][]): number {
  const queue: [number, number][] = [[0, 0]];
  const visited = new Set<string>();
  visited.add("0,0");
  let steps = 0;

  while (queue.length) {
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const [r, c] = queue.shift()!;
      if (r === targetR && c === targetC) return steps;
      for (const [dr, dc] of [[0,1],[0,-1],[1,0],[-1,0]]) {
        const nr = r + dr, nc = c + dc;
        const key = `${nr},${nc}`;
        if (valid(nr, nc) && !visited.has(key)) {
          visited.add(key);
          queue.push([nr, nc]);
        }
      }
    }
    steps++;
  }
  return -1;
}
```

DFS Template

```
function dfs(node: TreeNode | null): number {
  if (!node) return 0;
  const left = dfs(node.left);
  const right = dfs(node.right);
  return Math.max(left, right) + 1;
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	Number of Islands	Medium	DFS/BFS flood fill
2	Clone Graph	Medium	Map old→new nodes
3	Rotting Oranges	Medium	Multi-source BFS
4	Pacific Atlantic Water Flow	Medium	Reverse DFS from edges
5	Word Ladder	Hard	BFS shortest path

8. Pattern 6: Trees

When to use: Hierarchical data, BST properties, recursion, level-order traversal.

Traversals

```
// Inorder: left → node → right (BST gives sorted order)
function inorder(root: TreeNode | null, result: number[] = []): number[] {
  if (!root) return result;
  inorder(root.left, result);
  result.push(root.val);
  inorder(root.right, result);
  return result;
}

// Level order (BFS)
function levelOrder(root: TreeNode | null): number[][] {
  if (!root) return [];
  const result: number[][] = [];
  const queue = [root];
  while (queue.length) {
    const level: number[] = [];
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const node = queue.shift()!;
      level.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);
  }
  return result;
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	<u>Maximum Depth of Binary Tree</u>	Easy	DFS depth
2	<u>Validate Binary Search Tree</u>	Medium	Min/max bounds
3	<u>Lowest Common Ancestor of BST</u>	Medium	BST property
4	<u>Binary Tree Maximum Path Sum</u>	Hard	Global max + recursion
5	<u>Serialize and Deserialize Binary Tree</u>	Hard	BFS/DFS encoding

9. Pattern 7: Graphs

When to use: Dependencies, cycles, shortest path, topological sort, union-find.

Topological Sort (Kahn's BFS)

```
function topologicalSort(n: number, edges: number[][]): number[] {
  const indegree = Array(n).fill(0);
  const adj: number[][] = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adj[u].push(v);
    indegree[v]++;
  }
  const queue = indegree.map((d, i) => d === 0 ? i : -1).filter(i => i >= 0);
  const order: number[] = [];
  while (queue.length) {
    const node = queue.shift()!;
    order.push(node);
    for (const nei of adj[node]) {
      if (--indegree[nei] === 0) queue.push(nei);
    }
  }
  return order.length === n ? order : []; // empty if cycle
}
```

Union-Find

```
class UnionFind {
  parent: number[];
  rank: number[];
  constructor(n: number) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = Array(n).fill(0);
  }
  find(x: number): number {
    if (this.parent[x] !== x) this.parent[x] = this.find(this.parent[x]);
    return this.parent[x];
  }
  union(a: number, b: number): boolean {
    const ra = this.find(a), rb = this.find(b);
    if (ra === rb) return false;
    if (this.rank[ra] < this.rank[rb]) this.parent[ra] = rb;
    else if (this.rank[ra] > this.rank[rb]) this.parent[rb] = ra;
    else { this.parent[rb] = ra; this.rank[ra]++; }
    return true;
  }
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	Course Schedule	Medium	Cycle detection / topo sort
2	Course Schedule II	Medium	Return topo order
3	Network Delay Time	Medium	Dijkstra
4	Redundant Connection	Medium	Union-Find
5	Cheapest Flights Within K Stops	Medium	Bellman-Ford / BFS

10. Pattern 8: Dynamic Programming

When to use: Optimal substructure, overlapping subproblems, counting ways, min/max cost.

1D DP Template

```
function climbStairs(n: number): number {  
  if (n <= 2) return n;  
  let prev2 = 1, prev1 = 2;  
  for (let i = 3; i <= n; i++) {  
    const curr = prev1 + prev2;  
    prev2 = prev1;  
    prev1 = curr;  
  }  
  return prev1;  
}
```

2D DP Template

```
function uniquePaths(m: number, n: number): number {
  const dp = Array.from({ length: m }, () => Array(n).fill(1));
  for (let i = 1; i < m; i++) {
    for (let j = 1; j < n; j++) {
      dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
  }
  return dp[m - 1][n - 1];
}
```

DP Recognition Checklist

- Can problem be broken into smaller subproblems?
- Do subproblems repeat?
- Can you define `dp[i]` meaning clearly?
- What's the recurrence relation?

Problems (3+)

#	Problem	Difficulty	Key Insight
1	<u>Climbing Stairs</u>	Easy	Fibonacci pattern
2	<u>House Robber</u>	Medium	Take or skip
3	<u>Coin Change</u>	Medium	Unbounded knapsack
4	<u>Longest Increasing Subsequence</u>	Medium	DP + BS optimization
5	<u>Edit Distance</u>	Medium	2D string DP

11. Pattern 9: Heap / Priority Queue

When to use: K largest/smallest, merge K sorted, median stream, scheduling.

Template (Min-Heap for K Largest)

```
class MinHeap {
  private heap: number[] = [];
  push(val: number) {
    this.heap.push(val);
    this.bubbleUp(this.heap.length - 1);
  }
  pop(): number {
    const top = this.heap[0];
    const last = this.heap.pop()!;
    if (this.heap.length) {
      this.heap[0] = last;
      this.bubbleDown(0);
    }
    return top;
  }
  peek() { return this.heap[0]; }
  size() { return this.heap.length; }
  // bubbleUp, bubbleDown implementations...
}

function kthLargest(nums: number[], k: number): number {
  const heap = new MinHeap();
  for (const n of nums) {
    heap.push(n);
    if (heap.size() > k) heap.pop();
  }
  return heap.peek();
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	<u>Kth Largest Element in Array</u>	Medium	Min-heap size k
2	<u>Merge K Sorted Lists</u>	Hard	Heap of list heads
3	<u>Find Median from Data Stream</u>	Hard	Two heaps
4	<u>Task Scheduler</u>	Medium	Max-heap + cooldown
5	<u>Top K Frequent Words</u>	Medium	Heap with custom comparator

12. Pattern 10: Stack / Monotonic Stack

When to use: Valid parentheses, next greater element, histogram area, parsing.

Monotonic Stack Template

```
function dailyTemperatures(temps: number[]): number[] {  
  const result = Array(temps.length).fill(0);  
  const stack: number[] = []; // indices  
  
  for (let i = 0; i < temps.length; i++) {  
    while (stack.length && temps[i] > temps[stack[stack.length - 1]]) {  
      const idx = stack.pop();  
      result[idx] = i - idx;  
    }  
    stack.push(i);  
  }  
  return result;  
}
```

Problems (3+)

#	Problem	Difficulty	Key Insight
1	Valid Parentheses	Easy	Matching stack
2	Daily Temperatures	Medium	Monotonic decreasing
3	Largest Rectangle in Histogram	Hard	Monotonic + area calc
4	Evaluate Reverse Polish Notation	Medium	Stack evaluation
5	Car Fleet	Medium	Sort + stack

13. NeetCode 150 Mapping

Pattern	NeetCode Problems
Two Pointers	Valid Palindrome, Two Sum II, 3Sum, Container With Most Water, Trapping Rain Water
Sliding Window	Best Time Buy/Sell Stock, Longest Substring, Permutation in String, Min Window Substring
Hash Map	Two Sum, Group Anagrams, Top K Frequent, Product Except Self, Valid Sudoku
Binary Search	Binary Search, Search 2D Matrix, Koko Bananas, Find Min Rotated, Median Two Arrays
BFS/DFS	Number of Islands, Clone Graph, Rotting Oranges, Pacific Atlantic, Surrounded Regions
Trees	Invert Tree, Max Depth, Same Tree, Subtree, LCA BST, Validate BST, Kth Smallest
Graphs	Course Schedule I/II, Number of Connected Components, Redundant Connection
DP	Climbing Stairs, House Robber, Coin Change, LIS, LCS, Word Break, Decode Ways
Heap	Kth Largest, Merge K Lists, Find Median Stream, Task Scheduler
Stack	Valid Parentheses, Daily Temperatures, Largest Rectangle, Evaluate RPN

Full list: neetcode.io/practice

14. LeetCode Practice Plan

12-Week Plan (200 Problems)

Week	Focus	Target
1-2	Arrays, Hash Map, Two Pointers	30 easy/medium
3-4	Sliding Window, Binary Search	25 medium
5-6	Linked List, Stack, Heap	25 medium
7-8	Trees, BFS/DFS	30 medium
9-10	Graphs, Backtracking	25 medium
11-12	DP, Hard review	30 medium + 10 hard

Daily Routine (2 hours)

```
0:00-0:05  Review yesterday's problem
0:05-0:40  New problem (timed, no hints)
0:40-0:55  Watch/read solution if stuck
0:55-1:25  Second new problem
1:25-1:45  Re-implement without looking
1:45-2:00  Log patterns, update tracker
```

Spaced Repetition

- Day 1: Solve problem
- Day 3: Re-solve from scratch
- Day 7: Re-solve + explain aloud
- Day 30: Final review before interview

15. Interview Script

Opening (First 2 Minutes)

```
"Let me make sure I understand the problem..."  
[Restate problem, confirm inputs/outputs]  
"Can the input be empty? Are there duplicates? What's the expected size?"  
"Let me think of a few examples including edge cases."
```

Approach (Next 5 Minutes)

```
"Brute force would be  $O(n^2)$  by checking every pair..."  
"We can optimize using a hash map to  $O(n)$  time,  $O(n)$  space."  
"I'll use the two-pointer technique since the array is sorted."  
"Let me outline the algorithm step by step before coding."
```

Coding (15-20 Minutes)

- Use meaningful variable names
- Handle edge cases first
- Talk through logic as you write
- Don't panic on syntax — pseudocode if needed

Testing (5 Minutes)

```
"Let me trace through the example: [1,2,3], target=5"  
"Edge case: empty array → return []"  
"Edge case: no solution → return [-1,-1]"  
"Time:  $O(n)$ , Space:  $O(n)$  for the hash map"
```

When Stuck

```
"I'm considering two approaches: BFS vs DFS..."  
"For BFS, the trade-off is  $O(n)$  space for the queue..."  
"Can I get a hint on whether DP is the right direction?"
```

Common Follow-ups

- Optimize space to $O(1)$
- What if input doesn't fit in memory?
- How would you parallelize?
- Write unit tests for this function

Quick Reference

Two Pointers → sorted, pairs, palindrome
Sliding Window → contiguous subarray/substring
Hash Map → $O(1)$ lookup, frequency
Binary Search → sorted, minimize/maximize answer
BFS/DFS → graphs, grids, shortest path
Trees → recursion, BST, level order
Graphs → topo sort, union-find, Dijkstra
DP → optimal substructure, overlapping
Heap → k largest, merge k, median
Stack → matching, next greater, monotonic

Goal: Pattern recognition in < 3 minutes → correct approach → clean code → complexity analysis.

16. Product-company DSA (1 CR)

Generic patterns get you past startups. **Google / Meta / Amazon / Microsoft / Uber** grade *follow-ups* and *company flavor*. Full lists + loop shapes: [20-1cr-product-company-complete.pdf](#) §3–4.

What changes at 1 CR vs 30 LPA

30 LPA	1 CR product
One accepted solution	Brute → optimal → follow-up (stream / memory / threads)
Silence while coding	Narrate every 15-20 seconds
Only NeetCode 150	NeetCode 150 plus company-tagged + 1 unseen medium/week
Segment tree early	Skip segment tree until Phase 9 unless the problem forces it

Must-add families (do in Phase 6E-6F)

Family	Problems (do all)
Intervals	LC 56 Merge, 57 Insert, 253 Meeting Rooms II, 435 Non-overlap
Backtracking	LC 78 Subsets, 46 Permutations, 39 Combination Sum, 79 Word Search
Union-Find	LC 547 Provinces, 684 Redundant Connection, 721 Accounts Merge
Topo	LC 207 / 210 Course Schedule, 269 Alien Dictionary (premium or equivalent)
Design LC	LC 146 LRU, 355 Design Twitter, 380 Insert Delete GetRandom

Follow-up script (say after every accepted solution)

1. Trace a failing edge case.
2. Time / space; bottleneck at $n = 10^7$.
3. “If this is a stream...” (heap / two pointers).
4. “If two threads call this...” (lock vs snapshot).
5. Wrap as a small product API (`class Autocomplete`).

Company-tagged (first pass in Days 133-135)

- **Meta speed:** 56, 215, 236, 314, 347, 560, 680, 1249
- **Amazon:** 23, 42, 127, 146, 200, 238, 973, 994
- **Google think:** 42, 76, 124, 212, 295, 394
- **Uber graphs/heap:** 23, 253, 295, 973, 1091

Re-solve each **twice**, timed, then speak one follow-up.

17. Pattern 11: Intervals

When: overlapping ranges — meetings, calendar, video clips, merge slots.

Template: sort by start (sometimes end), then one pass merge or min-heap of end times.

```
function merge(intervals: number[][]): number[][] {
  intervals.sort((a, b) => a[0] - b[0]);
  const out = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const last = out[out.length - 1];
    if (intervals[i][0] <= last[1]) last[1] = Math.max(last[1], intervals[i][1]);
    else out.push(intervals[i]);
  }
  return out;
}
```

#	Problem	Why they ask
1	<u>Merge Intervals</u>	Meta / Amazon classic
2	<u>Insert Interval</u>	Same family, more branches
3	<u>Meeting Rooms II</u>	Heap of end times (Uber / Amazon)
4	<u>Non-overlapping Intervals</u>	Greedy by end

Follow-up they love: “Now do it for a calendar with recurring events” → treat expansion as a generator, still merge.

18. Pattern 12: Backtracking

When: enumerate subsets / permutations / constraint boards.

Template: choose → recurse → unchoose. Bound early.

```
function subsets(nums: number[]): number[][] {
  const res: number[][] = [];
  const path: number[] = [];
  function dfs(i: number) {
    if (i === nums.length) { res.push([...path]); return; }
    path.push(nums[i]); dfs(i + 1); path.pop();
    dfs(i + 1);
  }
  dfs(0);
  return res;
}
```

#	Problem	Note
1	Subsets	Template
2	Permutations	Used[] or swap
3	Combination Sum	Reuse allowed
4	Word Search	Grid + visited
5	N-Queens	Phase 9 / Google

Interview talk: state space size first (2^n subsets) so they know you will not TLE blindly.

Quick Reference (updated)

Two Pointers → sorted, pairs, palindrome
Sliding Window → contiguous subarray/substring
Hash Map → $O(1)$ lookup, frequency
Binary Search → sorted, minimize/maximize answer
BFS/DFS → graphs, grids, shortest path
Trees → recursion, BST, level order
Graphs → topo sort, union-find, Dijkstra
DP → optimal substructure, overlapping
Heap → k largest, merge k, median
Stack → matching, next greater, monotonic
Intervals → sort + merge / heap of ends
Backtracking → choose · recurse · unchoose

1 CR extra: after every problem, speak one follow-up. Lists: [20-1cr-product-company-complete.pdf](#) .

DSA Patterns Interview Q&A Bank – 100 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

50 interview questions with full detailed answers for two pointers, sliding window, hash map, binary search, Breadth-First Search (BFS), Depth-First Search (DFS), and dynamic programming intro.

Target companies: Google, Amazon, Flipkart, Uber, Microsoft, Meta.

How to use: Pick 2 questions daily. Read the question, answer aloud in 2–3 minutes, then check the full answer.

Q1: What is the two pointers pattern and when do you use it?

Answer: Two pointers use index `left` and `right` (or slow/fast) scanning array or string from different positions. Use on sorted arrays for pair sums, palindrome check, removing duplicates in-place, merging sorted arrays, and container with most water. Often $O(n)$ time $O(1)$ space vs $O(n^2)$ brute force. First check if sorting or sorted input enables two pointers.

Q2: How does two pointers work on a sorted array for pair sum?

Answer: Set `left=0`, `right=n-1`. If sum too small, `left++`; if too big, `right--`; if equal, record and move. $O(n)$ after $O(n \log n)$ sort, or $O(n)$ if already sorted. Works for 'two sum' in sorted array; unsorted version uses hash map. State why moving pointers is safe — monotonic reason.

Q3: What is the sliding window pattern?

Answer: Sliding window maintains a contiguous subarray/substring window with `left` and `right` expanding or shrinking while tracking window state (sum, frequency map, distinct count). Fixed window: size k slides. Variable window: expand until invalid, then shrink from left. $O(n)$ each element enters and leaves once. Classic for max sum subarray of size k , longest substring without repeating characters.

Q4: What is the difference between fixed and variable sliding window?

Answer: Fixed: window size k constant — compute first window, slide by add right remove left. Variable: grow right until constraint violated (sum $>$ target, duplicate char), then shrink left until valid again, track best answer at each valid state. Minimum window substring is variable window hard problem — Amazon favorite.

Q5: How does hash map pattern solve Two Sum?

Answer: One pass: for each `nums[i]`, check if `target - nums[i]` exists in map; if yes return indices; else store `nums[i] → i`. $O(n)$ time $O(n)$ space. Template for complement lookup, anagram grouping, frequency counting. Explain trade: $O(n)$ space buys $O(1)$ lookup vs sorting $O(n \log n)$ space $O(1)$.

Q6: When do you use hash map for frequency counting?

Answer: Count character or element frequencies: anagram detection, top K frequent, find duplicate, ransom note. Pattern: `map.set(c, (map.get(c) || 0) + 1)`. Compare maps or use single map with decrement for anagram. $O(n)$ time. Follow-up: array size n , values $1..n$ — use array as hash $O(1)$ space (pigeonhole).

Q7: Explain binary search beyond sorted arrays.

Answer: Binary search works on monotonic predicate: find first/last true in `[false,...,false,true,...,true]`. Examples: minimum capacity to ship packages, square root, search rotated sorted array (modified compare). Template: `while (lo < hi) { mid = lo + (hi-lo)/2; if (ok(mid)) hi = mid; else lo = mid+1; }`. $O(\log n)$ — always mention overflow-safe mid.

Q8: What is 'binary search on answer'?

Answer: When answer is in range $[lo, hi]$ and if value x works then all $x' > x$ work (monotonic), binary search the answer space without explicit array. Check feasibility with helper `can(x)`. Used in Koko eating bananas, split array largest sum. Interview script: define monotonic property before coding.

Q9: What is the BFS (Breadth-First Search) template?

Answer: Use queue. Enqueue start, mark visited. While queue: dequeue node, process, enqueue unvisited neighbors. Level-order: track queue size each iteration. Shortest path in unweighted graph — first time reaching target is minimum steps. $O(V+E)$ time. Never forget visited set or infinite loop on cycles.

Q10: What is the DFS (Depth-First Search) template?

Answer: Recursive: base case, mark visited, recurse neighbors. Iterative: stack push start. Use for exploring all paths, cycle detection, topological sort, connected components, island counting. $O(V+E)$. Backtracking is DFS with undo (remove from path after explore). Depth limit for large graphs to avoid stack overflow.

Q11: When do you choose BFS vs DFS?

Answer: BFS: shortest path unweighted, level-by-level, nearest layer (word ladder). DFS: enumerate paths, detect cycle with recursion stack colors, topological order, maze with one path. Memory: BFS $O(\text{width})$ queue; DFS $O(\text{depth})$ stack. Grid problems: both work for islands — BFS if shortest steps matter.

Q12: What is dynamic programming (DP) and when is it applicable?

Answer: DP solves problems with optimal substructure and overlapping subproblems by storing subproblem results. Applicable when brute force recursion revisits same states — Fibonacci, knapsack, longest increasing subsequence. Steps: define state, recurrence, base case, iteration order, space optimization. Say 'DP' only after identifying overlapping subproblems.

Q13: What is the difference between memoization and tabulation?

Answer: Memoization: top-down recursion + cache (hash map or array) — compute on demand. Tabulation: bottom-up fill table in known order — often no recursion stack. Same complexity usually. Tabulation easier to optimize space (rolling array). Interviews: start with memoization if recurrence natural, refactor to tabulation.

Q14: How do you define DP state in an interview?

Answer: State = parameters fully describing subproblem — e.g. `dp[i]` = best answer using first i items, `dp[i][w]` = max value with weight limit w . Wrong state = wrong solution. Write recurrence in words before code: `dp[i] = max(dp[i-1], nums[i] + dp[i-2])`. Validate with small example manually.

Q15: Explain the Kadane's algorithm pattern.

Answer: Maximum subarray sum: `currMax = max(nums[i], currMax + nums[i])`, track global max. $O(n)$ $O(1)$. DP view: $dp[i] = \text{max ending at } i$. Variants: return indices, circular array, max product (track min and max). Google easy-medium staple — know derivation not just memorization.

Q16: How does coin change DP work?

Answer: Minimum coins for amount: `dp[0]=0`, `dp[a] = min(dp[a-coin]+1)` for each coin. Unbounded knapsack order — iterate amounts outer loop. Count ways: `dp[a] += dp[a-coin]`. $O(\text{amount} \times \text{coins})$. Clarify unlimited vs 0/1 coin supply — changes inner loop direction.

Q17: What is the longest increasing subsequence (LIS) pattern?

Answer: $O(n^2)$ DP: `dp[i]` = LIS ending at i . $O(n \log n)$ with patience sorting — binary search on tails array. Patience sorting is advanced but impresses at Microsoft. Return length vs actual sequence differs — sequence needs parent pointers.

Q18: How do you solve grid BFS problems like rotten oranges?

Answer: Multi-source BFS: enqueue all sources (rotten oranges) with distance 0 simultaneously. Process level by level incrementing time. Shortest time for all cells. Template extends to 01 BFS (deque), fire spread, nearest gate. Mark visited when enqueue not dequeue to avoid duplicates.

Q19: What is topological sort and when is it needed?

Answer: Linear ordering of DAG (Directed Acyclic Graph) where every edge $u \rightarrow v$ has u before v . Kahn's algorithm: BFS with in-degree zero queue. Or DFS post-order reverse. Use for course

schedule, build order, dependency resolution. Cycle detection: if processed count $< V$, cycle exists — Flipkart dependency question.

Q20: What is backtracking and how differs from DP?

Answer: Backtracking explores all choices with undo — subsets, permutations, N-queens, sudoku. Exponential time often. Prune invalid branches early. DP when subproblems repeat and optimal structure; backtracking when count/enumerate all solutions. Combination sum with reuse may use backtracking; with optimization and overlap → DP.

Q21: Explain the subsets generation pattern.

Answer: For each element: include or exclude — recursion `backtrack(i, path)`. Or iterative bitmask 0 to $2^n - 1$. $O(2^n)$ output size unavoidable. Subsets II adds sort + skip duplicates at same level. Communicate time dominated by output size.

Q22: How do permutations differ from subsets in approach?

Answer: Permutations: use used array or swap, choose unused elements — $n!$ leaves. Subsets: ordered by index, no reuse of earlier index in same path. Next permutation (lexicographic) uses two-pointer swap algorithm — occasional follow-up.

Q23: What is the merge intervals pattern?

Answer: Sort by start time. Iterate: if current overlaps last merged ($\text{start} \leq \text{last.end}$), extend last.end; else push new interval. $O(n \log n)$. Used for meeting rooms, insert interval, employee free time. Overlap condition critical — clarify inclusive vs exclusive endpoints.

Q24: What is the fast and slow pointers pattern beyond linked lists?

Answer: Also tortoise-hare on arrays (find duplicate in $O(1)$ space — Floyd cycle in index array), middle of linked list, palindrome linked list (find mid, reverse second half). $O(n)$ one pass. Name pattern when interviewer sees two pointers moving at different speeds.

Q25: What is prefix sum and when to use it?

Answer: $\text{Prefix}[i] = \text{sum of nums}[0..i-1]$. Range sum query $O(1)$: $\text{prefix}[j+1] - \text{prefix}[i]$. 2D prefix for matrix region sum. Combine with hash map for subarray sum equals K — store prefix frequencies. $O(n)$ preprocessing, $O(1)$ or $O(n)$ queries depending variant.

Q26: When is greedy algorithm correct and how to justify?

Answer: Greedy picks locally optimal if problem has greedy choice property and optimal substructure — activity selection, non-overlapping intervals, Huffman coding. Prove or cite exchange argument. Wrong greedy on coin change $\{1,3,4\}$ target 6 — greedy $4+1+1=3$ coins, optimal $3+3=2$. Always test counterexample mentally.

Q27: How do you binary search a rotated sorted array?

Answer: Modified compare: if `nums[lo] <= nums[mid]` left half sorted, else right sorted. Check if target in sorted half, adjust lo/hi. $O(\log n)$. Duplicate values need `while (nums[hi] === nums[lo]) hi--` worst $O(n)$. Amazon rotation search is standard.

Q28: What is tree DP intro – example house robber III?

Answer: DP on tree: post-order compute `rob(node)` and `notRob(node)` from children returns. Cannot rob adjacent nodes. $O(n)$. Pattern: recurse children, combine at parent. Also diameter of tree, max path sum — state often includes/excludes current node.

Q29: How do you detect cycle in directed graph?

Answer: Three-color DFS: white unvisited, gray in current stack, black finished. Edge to gray node = cycle. Or Kahn topological sort — no complete order means cycle. Undirected: DFS track parent or Union-Find. State which graph type before answering.

Q30: Explain Dijkstra's algorithm at intro level.

Answer: Shortest path from source in weighted graph with non-negative edges. Min-heap stores (distance, node). Relax neighbors if shorter path found. $O((V+E) \log V)$. Cannot handle negative edges — use Bellman-Ford. Uber ETA maps to Dijkstra on road network — mention non-negative distances.

Q31: When do you use Union-Find in patterns?

Answer: Dynamic connectivity, count connected components after adding edges, Kruskal MST edge sorting + union, detect cycle in undirected graph on edge insert. Simpler than DFS when only group membership matters. $O(\alpha(n))$ nearly constant per operation.

Q32: What is the meeting rooms II pattern?

Answer: Min heap of end times: sort meetings by start, if start $\geq$ heap.min pop finished room, push end. Heap size = rooms needed. Greedy + heap. Variation of interval partitioning — classic calendar scheduling at Meta.

Q33: How does valid parentheses use stack pattern?

Answer: Push opening brackets. On closing, pop must match. $O(n)$. Extensions: generate valid strings, longest valid substring (stack stores indices), minimum remove to valid. Monotonic

stack variant for nested depth.

Q34: Explain longest substring without repeating characters.

Answer: Variable window + Set or Map of last index. Expand right, shrink left while duplicate. $O(n)$. Map last index allows jump left directly — optimization. Template for many 'substring with constraint' problems.

Q35: How do you approach 3Sum pattern?

Answer: Sort array. Fix i , two pointers on remainder for $-\text{nums}[i]$. Skip duplicates on i and pointers. $O(n^2)$. Extension 3Sum closest, 4Sum add another loop or hash. Communicate duplicate skipping to avoid double counting.

Q36: What is container with most water teaching?

Answer: Two pointers at ends. Move shorter line inward — only shorter limits height, moving taller cannot increase area. $O(n)$ greedy proof. Teaches monotonic pointer movement justification — interviewers want proof sketch.

Q37: What is binary tree level order traversal pattern?

Answer: BFS queue, track size per level. Variants: zigzag, right side view (last per level), vertical order (column + BFS). $O(n)$. Foundation for many tree problems before recursion-heavy ones.

Q38: How do you solve number of islands?

Answer: Grid DFS or BFS: for each '1' not visited, increment count, flood fill mark visited. $O(\text{rows} \times \text{cols})$. Union-Find alternative. Discuss in-place marking vs visited matrix — trade memory.

Q39: Explain climbing stairs as DP intro.

Answer: $\text{dp}[i] = \text{ways to reach step } i = \text{dp}[i-1] + \text{dp}[i-2]$. Fibonacci. $O(n)$ time $O(1)$ space with two variables. Introduces transition from recursion to DP. Generalizes with steps 1 or 2 or custom step sizes.

Q40: What is house robber linear DP?

Answer: Cannot rob adjacent houses: $\text{dp}[i] = \max(\text{dp}[i-1], \text{nums}[i] + \text{dp}[i-2])$. $O(n)$ $O(1)$. Circular variant needs split into two linear runs excluding first or last house. Classic state machine DP intro.

Q41: How does word break use DP?

Answer: $\text{dp}[i]$ = can segment $s[0..i]$? Check $j < i$ where $\text{dp}[j]$ and $s[j..i]$ in dictionary. $O(n^2 \times \text{dict lookup})$. BFS also works. Trie speeds dictionary lookup. Google medium — define boolean DP clearly.

Q42: How should you communicate approach in a coding interview?

Answer: Repeat problem, clarify inputs/outputs/edge cases, give brute force complexity, propose optimized pattern with why, get buy-in, code cleanly, test with example including edge case, state time/space. At Amazon use Leadership Principles tie-in sparingly. Silence while thinking is OK — narrate key insights.

Q43: How do you analyze time complexity for nested loops with two pointers?

Answer: Single loop with two pointers moving total n steps — $O(n)$ not $O(n^2)$. Sliding window each element enters and leaves once — $O(n)$. Nested binary searches $O(n \log n)$. State invariant maintaining why pointers only move forward.

Q44: What is space optimization in DP – rolling array?

Answer: If $dp[i]$ depends only on $dp[i-1]$ and $dp[i-2]$, keep two variables not full array — $O(1)$ space. 2D knapsack optimize to 1D array iterating weight backwards for 0/1 knapsack. Mention after correct 2D solution — shows senior polish.

Q45: When might bit manipulation appear with patterns?

Answer: XOR cancels duplicates — single number. Count bits, subset generation with bitmask. Usually secondary to main patterns but quick wins on easy Google questions. Know $x \& (x-1)$ removes lowest set bit.

Q46: Why sort as preprocessing step?

Answer: Enables two pointers, merge intervals, greedy scheduling, binary search on paired values. $O(n \log n)$ sort cost often dominates — still beats $O(n^2)$. Ask if sorting allowed or if original order must preserve.

Q47: Recursion vs iteration – interview preference?

Answer: Either works if correct. Iteration avoids stack overflow. Recursion clearer for trees and backtracking. Mention converting recursion to stack if depth large. Tail recursion not optimized

in JavaScript.

Q48: How do you pick a pattern when stuck?

Answer: Ask: sorted? → two pointers/binary search. Subarray/substring contiguous constraint? → sliding window. Need counts or complements? → hash map. Tree/graph? → BFS/DFS. Optimal substructure? → DP. Try brute force then optimize — pattern emerges from bottleneck.

Q49: What are common DSA interview mistakes?

Answer: Wrong complexity claim, missing empty input, off-by-one in binary search, not handling duplicates, infinite loop in while without progress, modifying array while iterating, forgetting visited in graph. Always walk through tiny example before coding.

Q50: Give a 60-second pattern cheat sheet for product company screens.

Answer: Two pointers: sorted pairs/in-place. Sliding window: contiguous subarray max/min. Hash map: frequency, complement, index. Binary search: sorted or monotonic answer. BFS: shortest steps. DFS: explore paths, cycles, components. DP: max/min/count ways with overlapping subproblems. State pattern name, complexity, and one example problem — interviewers score recognition speed at Google and Flipkart.

Questions 51-100 (top product companies)

Question 51: When is a monotonic stack the right tool?

Answer: Next greater/smaller in $O(n)$. Daily temperatures, histogram, trapping rain. Amazon / Google.

Question 52: Union-Find vs DFS for components?

Answer: Union-Find when you merge dynamically (accounts merge). DFS when the graph is static. Meta / Google.

Question 53: How do you detect a cycle in directed vs undirected graphs?

Answer: Directed: colors / recursion stack. Undirected: Union-Find or parent-aware DFS. Amazon.

Question 54: Explain topological sort Kahn vs DFS.

Answer: Kahn: queue of zero in-degree. DFS: finish times reverse. Course schedule. Microsoft / Amazon.

Question 55: What is a prefix sum used for?

Answer: Range sum in $O(1)$ after $O(n)$. Subarray sum equals K with a map. Meta.

Question 56: Binary search on rotated array – key insight?

Answer: One half is always sorted; decide which half holds the target. Microsoft / Amazon.

Question 57: How do you serialize a binary tree?

Answer: Preorder with null markers, or level order. LC 297. Amazon / Google.

Question 58: Trie vs hash map for prefix search?

Answer: Trie for prefixes and typeahead. Hash for exact. Google / Atlassian.

Question 59: Explain sliding window maximum.

Answer: Deque of decreasing indices; pop back if smaller, pop front if out of window. Hard Amazon.

Question 60: When is Dijkstra vs BFS?

Answer: BFS unweighted. Dijkstra non-negative weights. Uber maps.

Question 61: How do you find duplicates in $O(n)$ time $O(1)$ space if values are $1..n$?

Answer: Index marking or Floyd cycle (LC 287). Microsoft / Amazon.

Question 62: Kadane's algorithm in one minute.

Answer: Best subarray ending here = $\max(x, \text{best} + x)$. Max subarray. Adobe / Amazon.

Question 63: How do you merge K sorted lists?

Answer: Min-heap of heads. $O(N \log K)$. Google / Uber.

Question 64: Explain backtracking template.

Answer: Choose, recurse, unchoose. Bound early. Subsets / N-Queens. Google.

Question 65: What is a difference array?

Answer: Range increment in $O(1)$, then prefix. Corporate flight bookings. Google.

Question 66: How do you compute next permutation?

Answer: Find pivot, swap with next greater, reverse suffix. Microsoft.

Question 67: Explain 0/1 knapsack vs unbounded.

Answer: 0/1: each item once (loop weight backward). Unbounded: coin change (forward).
Amazon / Microsoft.

Question 68: Lowest Common Ancestor in a Binary Search Tree vs binary tree.

Answer: BST: walk from root. Tree: recurse both sides. Meta / Amazon.

Question 69: How do you find the median of two sorted arrays?

Answer: Binary search partition (hard). Google classic. Know the idea even if you code the merge in time.

Question 70: What is a sparse table used for?

Answer: Idempotent range queries (min) in $O(1)$ after $O(n \log n)$. Rare Google.

Question 71: Explain Dutch national flag.

Answer: Three pointers for 0/1/2 sort. Sort colors. Microsoft / Amazon.

Question 72: How do you clone a graph?

Answer: Hash old $\rightarrow$ new, BFS/DFS. LC 133. Amazon / Meta.

Question 73: What is a rolling hash (Rabin-Karp) for?

Answer: Substring search, plagiarism-ish. Occasional Google.

Question 74: Heap vs sorted array for top-K.

Answer: Heap $O(n \log K)$ better when $K \ll n$. Meta / Uber.

Question 75: How do you handle integer overflow in JS interviews?

Answer: JS Number is IEEE; use BigInt if they care. Mention $2^{53}-1$. Microsoft.

Question 76: Explain Morris traversal at a high level.

Answer: Thread null rights to successor; $O(1)$ space inorder. Rare Google flex — optional.

Question 77: When do you use two heaps?

Answer: Median stream, sliding window median. Google / Uber.

Question 78: How do you find an articulation point?

Answer: Tarjan / discovery times. Rare; say you would review if asked. Google L5.

Question 79: Explain bit tricks they actually ask.

Answer: $n \& (n-1)$ clear lowest one, xor for single number. Microsoft / Adobe.

Question 80: What is a Fenwick tree for?

Answer: Prefix sums with updates. Optional Phase 9. Google.

Question 81: How do you reconstruct a tree from inorder+preorder?

Answer: Preorder root, split inorder, recurse. Amazon / Microsoft.

Question 82: Explain greedy vs DP with an example.

Answer: Jump game II greedy; coin change DP when greedy fails. Amazon.

Question 83: How do you do matrix BFS (rotting oranges)?

Answer: Multi-source queue of all rotten, minutes = levels. Amazon.

Question 84: What is a monotonic deque vs stack?

Answer: Deque: both ends for window max. Stack: next greater. Amazon / Google.

Question 85: How do you count inversions?

Answer: Merge sort count. Occasional. Adobe / Google.

Question 86: Explain 2-sum variants in one breath.

Answer: Hash for unsorted, two pointers if sorted, 3-sum sort + two pointers. All screens.

Question 87: How do you find the longest palindromic substring approach?

Answer: Expand around center $O(n^2)$ is enough; Manacher is extra. Amazon / Microsoft.

Question 88: What is topological order used for in build systems?

Answer: Jobs with dependencies — same as course schedule. Uber / Microsoft.

Question 89: How do you random-pick an index with given weights?

Answer: Prefix sums + binary search (LC 528). Google / Meta.

Question 90: Explain flood fill vs islands.

Answer: Same DFS/BFS; flood fill is one component recolor. Amazon.

Question 91: When is recursion a no-hire?

Answer: No base case, $O(n)$ stack on a linked list when you could iterate. Amazon.

Question 92: How do you reverse a linked list in groups of K?

Answer: Count, reverse pointers, recurse/iterate rest. Amazon hard-medium.

Question 93: What is a window + freq map pattern?

Answer: Anagrams, min window substring. Meta / Amazon.

Question 94: How do you explain time complexity of quicksort worst case?

Answer: Already sorted + bad pivot = n^2 ; randomize or median-of-three. Microsoft.

Question 95: Design Twitter (LC 355) structures.

Answer: User→tweets, follow map, merge K latest. Meta / Amazon design-LC.

Question 96: How do you find a duplicate and missing number?

Answer: Xor or sum vs expected, or index mark. Microsoft.

Question 97: Explain BFS shortest path on a grid with obstacles.

Answer: Queue, visited, sometimes 0-1 BFS if two edge weights. Uber / Google.

Question 98: What is a greedy interval schedule?

Answer: Sort by end, pick non-overlapping. Meeting rooms contrast. Amazon / Uber.

Question 99: How do you test your solution in 2 minutes?

Answer: Empty, one, duplicates, already sorted, max n. All.

Question 100: What follow-up do you always offer?

Answer: Stream, disk-too-big, concurrent callers. Google L5. PDF #20.